

Mauricio Salinas

Back-end Developer · Python · FastAPI · Microservicios Event-Driven · Clean Architecture

Santiago, Chile · maundrex@gmail.com · +56 9 4758 6414

github.com/MauriDev94 · linkedin.com/in/mauricio-salinas-rubio · Inglés C2 (EF SET 71/100) · Disponible para trabajo remoto

PERFIL

Desarrollador backend enfocado en **Python**, **APIs REST** y **microservicios event-driven**. Construyo servicios con **FastAPI** y **Django/DRF** aplicando **SOLID**, **Clean Architecture**, **DDD** y **TDD** — especificación primero, implementación con cobertura de tests real. Experiencia en **PostgreSQL**, **mensajería con RabbitMQ** (idempotencia, dead-letter queues, eventual consistency), autenticación avanzada (JWT/OAuth2/OTP/SSO), **observabilidad** (logging JSON + correlation IDs) y pipelines de **CI/CD con GitHub Actions**. Inglés C2 para documentación técnica y trabajo con equipos remotos.

EXPERIENCIA

Desarrollo Backend — Proyectos de Portafolio (Independiente)

Santiago, Chile · Abr 2025 – presente

Construcción de APIs REST production-grade como portafolio profesional, con foco en patrones que aparecen en entrevistas backend senior.

- Diseño y desarrollo de APIs REST con **FastAPI** y **Django/DRF** aplicando Clean Architecture, SOLID, DDD y TDD.
- Diseño de **sistemas event-driven con microservicios** (RabbitMQ): idempotencia, dead-letter queues, retries con backoff, eventual consistency y correlation IDs end-to-end.
- Autenticación avanzada: **JWT**, **OAuth2**, **OTP**, **reset de contraseña** y **Google login vía SSO**.
- Optimización de consultas ORM (select_related/prefetch_related, cero N+1), transacciones atómicas y prevención de *race conditions*.
- Control de acceso basado en roles (**RBAC**) a nivel de objeto y **rate limiting** anti fuerza-bruta.
- Middleware transversal: logging estructurado, manejo de errores y seguridad.
- Pipelines de **CI/CD** con GitHub Actions (lint, tests automatizados, cobertura, reporte a Codecov) y despliegue continuo en Render.
- Contenerización con **Docker** y diseño de esquemas **PostgreSQL** con migraciones.

Analista de Soporte

Cadaques (Domino) · Santiago, Chile · Abr 2023 – Jul 2024

- Gestión y mantenimiento de bases de datos **SQL**; reportes estratégicos para operaciones.
- Desarrollo de **scripts Python** para automatización de procesos operativos, reduciendo trabajo manual repetitivo.
- Monitoreo y configuración de infraestructura de red empresarial.

PROYECTOS

Event-Driven Orders — Microservicios con RabbitMQ

Python 3.12 · FastAPI · RabbitMQ (aio-pika) · PostgreSQL 16 · Pydantic v2 · Docker Compose · pytest · testcontainers · structlog · GitHub Actions

Sistema de procesamiento de órdenes e-commerce con **3 microservicios** (order, inventory, notification) que se comunican exclusivamente por **eventos** vía RabbitMQ, aplicando Clean Architecture + DDD en cada servicio.

- **Idempotencia** en los consumers (processed_events con INSERT ... ON CONFLICT DO NOTHING): un evento redelivered no duplica efectos de negocio.
- **Atomicidad anti race-condition** en la reserva de stock (UPDATE condicional + row-level locking + SAVEPOINT).
- **Dead-letter queues + retries con backoff** (3 etapas) con clasificación de fallos transitorios/permanentes.
- **Eventual consistency** por coreografía de eventos, sin transacción distribuida.
- **Observabilidad**: logging estructurado JSON + **correlation ID end-to-end** (HTTP → eventos → 3 servicios) y reconexión robusta al broker.
- **database-per-service** (Postgres por servicio); **138 tests** con testcontainers (PostgreSQL real).
- **CI/CD**: GitHub Actions con matrix por servicio (lint + mypy + tests).

Repo: github.com/MauriDev94/event-driven-orders

MedBook API — Reservas médicas con Django + DRF

Python 3.12 · Django 5.1 · DRF · PostgreSQL 16 · simplejwt · drf-spectacular · pytest · factory-boy · Docker · GitHub Actions · Codecov

API REST para gestión de citas médicas que cubre el ciclo completo de agendamiento: los médicos publican horarios, el sistema genera slots concretos y los pacientes los reservan, avanzando por una **máquina de estados** (pending → confirmed → completed / cancelled / no-show).

- **274 tests · 98.7% branch coverage · 0 fallos** — desarrollado con **TDD estricto** (Red-Green-Refactor) y `--cov-fail-under` enforced en CI.
- **RBAC** con 8 clases de permisos y control de acceso por rol (médico/paciente/admin) a nivel de objeto.
- **Transacciones atómicas + UPDATE atómico** para prevenir *race conditions* al reservar slots concurrentes.
- Optimización ORM: `select_related/prefetch_related` (cero N+1), `annotate()/Q()` para queries.
- Documentación **OpenAPI 3** con drf-spectacular (Swagger UI) y notificaciones por email (django-anymail/Resend).
- **Rate limiting** (AnonRateThrottle) para protección anti fuerza-bruta en login.
- **CI/CD**: GitHub Actions con lint (ruff), tests contra PostgreSQL real y cobertura a Codecov.

Repo: github.com/MauriDev94/MedBook · Live (Swagger UI): medbook-oiiio.onrender.com/api/docs

API Monolith — Monolito modular con Clean Architecture

Python · FastAPI · Django · PostgreSQL · SQLAlchemy · Alembic · Pytest · Docker · JWT · OAuth2 · OTP · SSO · GitHub Actions · Render

API REST construida con **FastAPI** y Django aplicando Clean Architecture, SOLID y DDD. Módulos independientes: **Auth, Users, Todos y Notifications**. Auth incluye JWT/OAuth2, OTP, reset de contraseña y Google login vía SSO.

- Middleware para logging, manejo de errores y seguridad transversal.
- Esquemas PostgreSQL con **SQLAlchemy** y migraciones con **Alembic**.
- Pipeline **CI/CD** con GitHub Actions: lint (ruff, black), suite Pytest con cobertura mínima del 70%.
- **Desplegada en producción en Render**.

Repo: github.com/MauriDev94/Api_monolith · Live (Swagger UI): api-monolith.onrender.com/docs

HABILIDADES TÉCNICAS

- **Lenguajes**: Python, SQL
- **Frameworks**: FastAPI, Django, Django REST Framework (DRF)
- **Mensajería / event-driven**: RabbitMQ (aio-pika), idempotencia, dead-letter queues, retries con backoff, eventual consistency
- **Bases de datos**: PostgreSQL, MongoDB — diseño de esquemas y consultas, optimización ORM (cero N+1)
- **ORM y migraciones**: SQLAlchemy, Alembic, Django ORM
- **Testing**: Pytest, pytest-django, TDD, factory-boy, testcontainers, pruebas unitarias e integración, coverage.py
- **Auth y seguridad**: JWT, OAuth2, OTP, SSO, Google Login, RBAC, rate limiting (throttling)
- **API**: REST, OpenAPI 3 / Swagger (drf-spectacular), diseño de endpoints, versionado
- **CI/CD**: GitHub Actions (lint, tests, cobertura, artefactos), Codecov, pre-commit
- **DevOps**: Docker, Git, GitHub, Render (despliegue continuo)
- **Observabilidad**: structlog (logging JSON), correlation IDs end-to-end
- **Calidad y metodologías**: SOLID, patrones de diseño, Clean Code, ruff, black · Clean Architecture, DDD, TDD, SDD, Microservicios event-driven, database-per-service, Modular Monolith

IDIOMAS

- **Español**: Nativo
- **Inglés: C2 Proficient** — **EF SET 71/100** (documentación técnica y comunicación con equipos remotos)

EDUCACIÓN

Analista Programador — Instituto Profesional Santo Tomás · Santiago · Mar 2021 – Jul 2025